

Web Services Technologies Course

Summary

Slide 1: Requirements of an IT Project

- **Functional Requirements:** These define *what* the application does to meet business needs.
- **Technical Requirements:** These define *how* the system performs. Key pillars include:
 - **Performance:** Focuses on fast response times and high availability (fault tolerance).
 - **Scalability:** Avoiding "scaling up" issues as demand grows.
 - **Maintainability:** Ensuring the app can evolve without breaking (Open/Closed Principle).
 - **Interoperability:** The ability to communicate with remote apps and serve diverse clients (Mobile, Desktop, SMS).
 - **Security & Portability:** Protecting data and ensuring the app works across different environments.

Slide 2: Constat (The Reality of Development)

This slide highlights that building modern IT systems from scratch is too difficult. Instead, developers leverage existing frameworks and servers—essentially "using the experience of others" to meet technical requirements.

Key Tools & Categories:

- **Application Servers:** The "home" where your app lives (e.g., Tomcat for web, JBoss for full JEE).
- **IoC (Inversion of Control):** Frameworks like **Spring** (light) or **EJB** (heavy) that manage how different parts of your code connect.
- **ORM (Object-Relational Mapping):** Tools like **Hibernate** or **JPA** that allow Java code to "talk" to databases easily.
- **Web Frameworks:** Standard ways to build web interfaces (e.g., Spring MVC).
- **Middleware:** The "glue" for communication.
 - **JAX-WS:** Used for SOAP Web Services.
 - **JAX-RS:** Used for RESTful Web Services.
 - **JMS:** Used for sending messages between applications.

Slide 3: Global Vision of a Distributed Architecture

This slide provides a visual map of how modern applications are structured internally and how they communicate with the outside world. It shows two separate entities (**Enterprise 1** and **Enterprise 2**) that need to exchange information.

1. Internal Application Layers

Inside each Application Server, the software is divided into specific layers to stay organized:

- **DAO (Data Access) Layer:** This layer is responsible for talking directly to the database (**SGBD**).
- **Business Layer:** This is the "brain" of the application where the main logic and rules are processed.
- **Web Layer:** This handles requests coming from web users (**Client HTTP**).
- **Service Layer:** This is the gateway for other applications. It uses **Middlewares** to allow remote communication.

2. Middleware & Communication

To allow "Enterprise 1" to talk to "Enterprise 2," they use specific technologies called **Middlewares**:

- **RMI & CORBA:** Older standards for distributed applications.
- **JMS:** For messaging between systems.
- **Web Services:** The modern standard for connecting different systems.
- **Network Protocols:** These systems physically connect using **TCP/IP** or **UDP**.

Slide 4: Architecture J2EE

This slide provides a detailed look at the internal components of a **J2EE Application Server** and how different clients interact with it.

1. Client Types & Communication Protocols

The architecture supports a wide variety of clients, each using a specific communication method:

- **Client Java:** Uses **RMI** (Remote Method Invocation) or **JMS** (Java Message Service) to talk directly to the business layer.
- **Client HTTP:** Standard web browsers using **HTTP/HTML** to access web pages.

- **Client SOAP:** Applications (written in Java, .NET, PHP, etc.) using **SOAP/XML** to exchange structured data.
- **Client HTTP (REST):** Modern clients like **Mobile apps** or JavaScript frameworks (jQuery) using **JSON or XML** via RESTful services.

2. Inside the Application Server

The server is divided into specialized containers and services:

- **Web Container (Web Layer):** Manages the presentation and service entry points.
 - Includes **Servlets/JSP** (using frameworks like Spring MVC or JSF).
 - Hosts **SOAP Services** (using AXIS or CXF) and **RESTful Services** (using Jersey or CXF).
- **Spring or EJB Container (Business Layer):** The core "brain" where **Business Components** (Composants Métier) and **Entities** (data objects) reside.
- **Infrastructure Services:** The "tools" that support the application:
 - **Connectivity:** Jax WS and Jax RS for web services.
 - **Management:** JTA (Transactions) and JNDI (Naming/Directory).
 - **Persistence:** JPA, Hibernate, and JDBC act as the bridge to the **Database (SGBD)**.

Slide 6: Introduction to Web Services

This slide introduces the fundamental concept of Web Services as the modern solution for connecting different software systems.

1. What is a Web Service?

- **Definition:** Web Services are web components based on the **Internet (HTTP)** that perform specific tasks.
- **Standardized Format:** They follow specific formats, traditionally **XML**. (Note: As seen in Slide 4, modern services also frequently use **JSON**).
- **Remote Access:** They allow an application to "call" or use a function that is hosted on a completely different server, simplifying how data is shared between companies.

2. Key Advantage: Universal Communication

The most important feature of Web Services is their **Interoperability**. They allow applications to communicate regardless of:

- **Execution Platform:** (e.g., Linux, Windows, or Cloud).
- **Implementation Language:** (e.g., a Java application can talk to a program written in PHP, .NET, or Cobol).

3. Evolution from CORBA

Web Services are the "next step" in evolution for distributed systems. They follow earlier initiatives like **CORBA** (Common Object Request Broker Architecture) but are:

- **Simpler:** They are easier to build and maintain.
- **Standardized:** They use recognized technologies (like HTTP) that are now accepted by the entire tech industry.

Slide 7: The HTTP Protocol (Le Protocole HTTP)

This slide focuses on the backbone of web communication: **HTTP** (HyperText Transfer Protocol). It explains how clients interact with servers to retrieve information.

1. What is HTTP?

- **Definition:** A protocol that allows a **client** (like a web browser or a mobile app) to retrieve documents from a **server**.
- **Types of Documents:**
 - **Static Content:** Files that do not change, such as standard **HTML** pages, **PDFs**, or **Images**.
 - **Dynamic Content:** Content generated in real-time by the server when requested, using technologies like **PHP**, **JSP**, or **ASP**.
- **Interaction:** Beyond just reading data, HTTP also allows the **submission of forms** (e.g., sending login details or registration data to a server).

2. How it Works (HTTP/1.0 Operation)

The process follows four simple steps:

1. **Connection:** The client establishes a connection to the server (creates a **socket**).
2. **Request:** The client sends an **HTTP request** for a specific document.
3. **Response:** The server sends back the requested document.
 - **Status 200:** Success! The document is returned.
 - **Status 404:** Error! The document was not found.
4. **Disconnection:** The client logs out or the connection is closed.

Slide 8: HTTP Protocol Methods

This slide explores the various "**verbs**" or **methods** used in an HTTP request. These methods tell the server exactly what action the client wants to perform on a specific resource or document.

1. The Core HTTP Methods

- **GET:** Used to **retrieve or recover** the content of a document from the server. This is the most common method used when you type a URL into a browser.
- **POST:** Used to **submit data** to the server, typically through a form. The data entered by the user is sent within the body of the request.
- **PUT:** Used to **upload or send a file** from the client to the server.
- **DELETE:** Used to request that the server **remove or delete** a specific document.
- **HEAD:** Used to **retrieve metadata** (information about a document) without downloading the actual content. This includes the document's type, size (capacity), and the date it was last modified.

Slide 9: Structure of an HTTP POST Request

This slide provides a concrete example of how an HTTP request is actually structured when using the **POST method**. It illustrates the communication between a **Client HTTP** and an **Application Server**.

1. The Request Header

The header contains metadata about the request. In this example:

- **Request Line:** Post /Nom_Script HTTP/1.0
 - **Method:** POST (Telling the server we are submitting data).
 - **Resource:** /Nom_Script (The specific program or script on the server to handle the data).
 - **Protocol:** HTTP/1.0 (The version being used).
- **Accept:** text/html (Tells the server the client wants to receive an HTML document in return).
- **Accept-Language:** fr (Requests the content in French).
- **User-Agent:** Mozilla/4.0 (Identifies the client software, in this case, an older version of a web browser).

2. The Critical Line Break

Notice the "*** line break ***". In the HTTP protocol, a blank line is mandatory to separate the **Header** (the instructions) from the **Body** (the actual data).

3. The Request Body

Unlike the **GET** method (where data is often attached to the URL), the **POST** method places user data in the **Body**:

- **Content:** login=Value1& pass=Value2 & Var3=Value3
- This represents key-value pairs (like a username and password) sent from a form.

Slide 10: Structure of an HTTP GET Request

This slide illustrates the structure of an HTTP request using the **GET method**, which is the primary way a **Client HTTP** retrieves information from a server,.

1. The GET Request Line

- **Method:** GET
- **Query String:** /Nom_Script?login=val1&pass=val2&...
 - Unlike the POST method where data is hidden in the body, the GET method attaches user data (like login and pass) directly to the **URL** using a question mark (?).
 - Each piece of data is a "key=value" pair separated by an ampersand (&).
- **Protocol:** HTTP/1.0

2. The Request Header

Just like in the POST example, the header tells the server about the client's capabilities:

- **Accept:** text/html (The client wants a web page).
- **Accept-Language:** fr (The client prefers French).
- **User-Agent:** Mozilla/4.0 (The client is a web browser).

3. The Request Body

- **Status:** Empty.
- Because the GET method sends all its data through the URL in the header, there is no need for a message body.

Slide 11: Server Returns Response

This slide shows the final part of the HTTP communication cycle: the **Response**. After the server processes the request (GET or POST), it sends back a structured message to the client.

1. The Response Header

This section contains metadata about the response being sent back:

- **Status Line:** HTTP/1.0 200 OK
 - **Protocol:** HTTP/1.0
 - **Status Code:** 200 (This means the request was successful).
 - **Status Message:** OK
- **Date:** The exact time the response was generated.
- **Server:** Apache/1.3.24 (Identifies the software the server is running).
- **Last-Modified:** Shows when the document was last updated, which helps with caching.
- **Content-Type:** Text/html (Tells the client to expect a web page/HTML).
- **Content-Length:** 4205 (The size of the message body in bytes).

2. The Mandatory Line Break

Just like in the request, there is a **blank line** that separates the instructions (Header) from the actual content (Body).

3. The Response Body

This contains the information the user actually sees:

- **Content:** <HTML><HEAD> </BODY></HTML>
- This is the code that the browser (Client HTTP) will render into a visual web page.

Slide 12: The Idea of Web Services (Concise Summary)

The core idea is to allow **different software systems** to talk to each other using a universal "language" (XML) over the internet (HTTP).

1. Main Concepts:

- **Interoperability:** A server can provide services to any client, regardless of its language (PHP, Java, .NET, or even Cobol).
- **Request/Response Cycle:**
 - **Request:** The client sends an **XML Envelope** containing a command (e.g., conversion) and data (e.g., montant=12).
 - **Response:** The server processes the logic and sends back an **XML Envelope** with the result (e.g., return=132).
- **Server Technologies:**
 - **Jax WS:** The engine that runs the web service.
 - **JaxB:** The tool that converts Java data (like an account "Compte") into XML format.

Slide 17: Concepts of Web Services

This slide outlines the three "pillars" or core standards that make web services function: **SOAP**, **WSDL**, and **UDDI**. These three concepts work together to help applications find each other, understand what they do, and exchange data.

1. Professional Summary: The Three Pillars

- **SOAP (Simple Object Access Protocol):**
 - **What it is:** The communication protocol (the "language") used for exchanging data between applications.
 - **How it works:** It is based on **XML** and is completely **platform-independent**.
 - **The Technical Part:** A SOAP call is an ASCII stream (text) wrapped in XML tags (like the <Envelope> seen in **Slide 12**) and is transported using the **HTTP protocol**.
- **WSDL (Web Services Description Language):**
 - **What it is:** The "technical manual" or **description** of a web service in XML format.
 - **Role:** it tells a client exactly what **methods** can be used, their **signatures** (what data they need), and the **access point** (the URL or port where the service lives).
 - **Comparison:** It is the modern equivalent of the **IDL** language used in older CORBA programming.
- **UDDI (Universal Description, Discovery and Integration):**
 - **What it is:** A global **directory** or "yellow pages" for web services.
 - **Role:** It allows companies to **publish** their services so other applications can **search** for and find them.
 - **Functionality:** Interestingly, UDDI itself acts like a web service; you use **SOAP** to search the UDDI directory.

Slide 19: SOAP (Simple Object Access Protocol)

This slide focuses on **SOAP**, the primary protocol used to invoke methods on remote services.

1. Professional Summary: Machine-to-Machine Communication

- **Definition:** SOAP is a protocol designed for **method invocation** on remote services, meaning it allows one computer to "call" a function on another computer.
- **Foundation:** It is entirely based on **XML**, ensuring that the data format is structured and readable by different types of systems.
- **Primary Objective:** Its main goal is to ensure seamless **communication between machines**, regardless of their internal hardware or software.
- **Mechanism:** It uses a transport protocol, typically **HTTP**, to send messages and perform **RPC (Remote Procedure Call)**. This allows a client to trigger a specific process (like a calculation or data retrieval) on a remote server as if it were running locally.
- **Network of Information:** SOAP is highly effective for web services because it can aggregate and provide a client with large amounts of information gathered from multiple **third-party servers** across a network.

Slides 20 & 21: Structure of a SOAP Message

These slides detail the four internal components of a **SOAP message**. This structure is what allows a **Client SOAP** (like Java, .NET, or Cobol) to send instructions to a **J2EE Application Server**.

- **SOAP Envelope:** This is the **basic and mandatory element** of every SOAP message.
 - It acts as the root of the XML document.
 - It defines the **namespaces** (to avoid naming conflicts) and the **data encoding** rules used in the message.
- **SOAP Header:** This is an **optional** part of the message.
 - It allows developers to add extra features (like security tokens or routing information) in a decentralized way.
 - It is specifically useful when a message must pass through and be processed by several **intermediary servers** before reaching its final destination.
- **SOAP Body:** This is the container for the actual information intended for the recipient.
 - It contains the specific **methods and parameters** the client wants the server to execute.
 - For example, in a banking service, the body would contain the instruction to "convert" a specific "amount".
- **SOAP Fault:** This is an **optional element** located inside the SOAP Body.
 - It is used strictly for **reporting errors** that occurred during the processing of the message.

Slide 23

JAX-WS (Java API for XML Web Services) – Summary

1. Definition

JAX-WS is a Java API used to **develop SOAP-based web services**.

It is the **successor of JAX-RPC**, offering a simpler and more modern approach.

2. Main Purpose

It allows developers to **create web services easily in Java** without dealing directly with complex XML or SOAP messages.

3. Annotations-Based Approach

- JAX-WS uses **annotations** (like @WebService, @WebMethod)
- These annotations define how a Java class is exposed as a web service
- This simplifies the **Java-to-WSDL mapping**

4. Automatic WSDL Generation

- The **WSDL file is generated automatically**
- This happens **during deployment** by the application server
- No need to manually write the WSDL

5. Key Advantage

- Reduces complexity
- Speeds up development
- Enables **easy integration between systems using XML-based communication**

Slide 24 :

JAX-WS / JAXB – Summary

1. Relationship Between JAX-WS and JAXB

- JAX-WS uses **JAXB (Java Architecture for XML Binding)** for handling data conversion
- JAXB is responsible for **mapping between Java objects and XML**

2. Main Role of JAXB 2.0

JAXB provides two key operations:

- **Marshalling**: Convert Java objects → XML
- **Unmarshalling**: Convert XML → Java objects

3. Schema-to-Code and Code-to-Schema

- JAXB can **generate Java classes from an XML Schema (XSD)**
- It can also **generate an XML Schema from Java classes**

4. Why It Matters in JAX-WS

- Web services communicate using **XML**
- JAXB makes it easy to **automatically convert data** between Java and XML without manual parsing

Slide 25 :

Principle of JAXB – Summary

1. **Unmarshalling (XML → Java Objects)**
 - Converts an **XML document into Java objects**
 - Used when receiving data from a web service and turning it into usable Java data
2. **Marshalling (Java Objects → XML)**
 - Converts **Java objects into an XML document**
 - Used when sending data from a Java application to a web service

Slide 26 : Marshalling (Java Object → XML)

```
1  package ws;
2
3  import java.io.File;
4  import java.util.Date;
5  import javax.xml.bind.JAXBContext;
6  import javax.xml.bind.Marshaller;
7
8  public class Banque {
9
10     public static void main(String[] args) throws Exception {
11
12         // Create JAXB context for the Compte class
13         JAXBContext context = JAXBContext.newInstance(Compte.class);
14
15         // Create a Marshaller object (used for Java → XML)
16         Marshaller marshaller = context.createMarshaller();
17
18         // Format the generated XML for better readability
19         marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
20
21         // Create a Java object
22         Compte cp = new Compte(1, 8000, new Date());
23
24         // Convert the Java object into an XML file
25         marshaller.marshal(cp, new File("comptes.xml"));
26
27         System.out.println("XML file generated successfully!");
28     }
29 }
```

Slide 27: Unmarshalling (XML → Java Object)

```
1 package ws;
2
3 import java.io.File;
4 import javax.xml.bind.JAXBContext;
5 import javax.xml.bind.Unmarshaller;
6
7 public class Banque2 {
8
9     public static void main(String[] args) throws Exception {
10
11         // Create JAXB context for the Compte class
12         JAXBContext context = JAXBContext.newInstance(Compte.class);
13
14         // Create an Unmarshaller object (used for XML → Java)
15         Unmarshaller unmarshaller = context.createUnmarshaller();
16
17         // Read the XML file and convert it into a Java object
18         Compte cp = (Compte) unmarshaller.unmarshal(new File("comptes.xml"));
19
20         // Display the object data
21         System.out.println("Code: " + cp.getCode());
22         System.out.println("Solde: " + cp.getSolde());
23         System.out.println("Date Creation: " + cp.getDateCreation());
24     }
25 }
```

Slides 28-29-30-31-32 :

Generating Classes from XML Schema (JAXB + XJC)

Idea

- JAXB allows you to **convert an XML Schema (XSD) into Java classes**
- This makes XML data easier to use inside Java programs

```
1  package ws;
2
3  import java.io.File;
4  import java.io.IOException;
5
6  import javax.xml.bind.JAXBContext;
7  import javax.xml.bind.SchemaOutputResolver;
8
9  import javax.xml.transform.Result;
10 import javax.xml.transform.stream.StreamResult;
11
12 public class Banque {
13
14     public static void main(String[] args) throws Exception {
15
16         // Create JAXB context for the Compte class
17         JAXBContext context = JAXBContext.newInstance(Compte.class);
18
19         // Generate XML Schema (XSD) from Java class
20         context.generateSchema(new SchemaOutputResolver() {
21
22             @Override
23             public Result createOutput(String namespaceUri, String suggestedFileName)
24                 throws IOException {
25
26                 // Define output file for the schema
27                 File file = new File("compte.xsd");
28
29                 // Create result stream pointing to the file
30                 StreamResult result = new StreamResult(file);
31
32                 // Set file name as system identifier
33                 result.setSystemId(file.getName());
34
35                 return result;
36             }
37         });
38
39         System.out.println("XSD schema generated successfully!");
40     }
41 }
```

2. How to Develop a JAX-WS Web Service

Step 1: Create the Web Service

- Develop the service using Java + annotations
- Deploy it on:
 - An **HTTP Server**
 - A **Web Service Container** such as:
 - JAX-WS
 - Apache Axis
 - Apache CXF

Step 2: Test the Web Service

- Use SOAP testing tools like:
 - SoapUI
 - Oxygen XML
- Purpose:
 - Send requests
 - Check responses
 - Debug the service

Step 3: Create Clients

Different clients can consume the web service:

1. **Java Client**
2. **.NET Client**
3. **PHP Client**

👉 Because web services are **platform-independent**

3. Some JAXB annotations

Annotation	Description
@XmlRootElement	Associates a class or enumeration with an XML root element
@XmlSchema	Associates an XML namespace with a Java package
@XmlTransient	Excludes a field/property from being mapped to XML
@XmlAttribute	Maps a Java property to an XML attribute
@XmlElement	Maps a Java property to an XML element
@XmlAccessorType	Defines how fields/properties are accessed and serialized in XML
@XmlNs	Associates a namespace prefix with a URI

Application Part:

Create a java web service using JaxWS which allows you to:

- **Convert an amount from Euro to DH**
- **Consult an account giving its code**
- **Consult a list of accounts**
- **Test the web service with a SOAP analyzer**

Step 1: Create the class

```
1 package jaxws2;
2
3 import java.util.Date;
4 import javax.xml.bind.annotation.XmlRootElement;
5
6 @XmlRootElement
7 public class Compte {
8
9     private long id;
10    private double solde;
11    private Date dateCreation;
12
13    public Compte() {
14    }
15
16    public Compte(long id, double s, Date date) {
17        this.id = id;
18        this.solde = s;
19        this.dateCreation = date;
20    }
21
22    public long getId() {
23        return this.id;
24    }
25
26    public double getSolde() {
27        return this.solde;
28    }
29
30    public Date getDate() {
31        return this.dateCreation;
32    }
33
34    public void setID(long idtoSet) {
35        this.id = idtoSet;
36    }
37
38    public void setSolde(double SoldeToSet) {
39        this.solde = SoldeToSet;
40    }
41
42    public void SetDate(Date DatetoSet) {
43        this.dateCreation = DatetoSet;
44    }
45 }
```

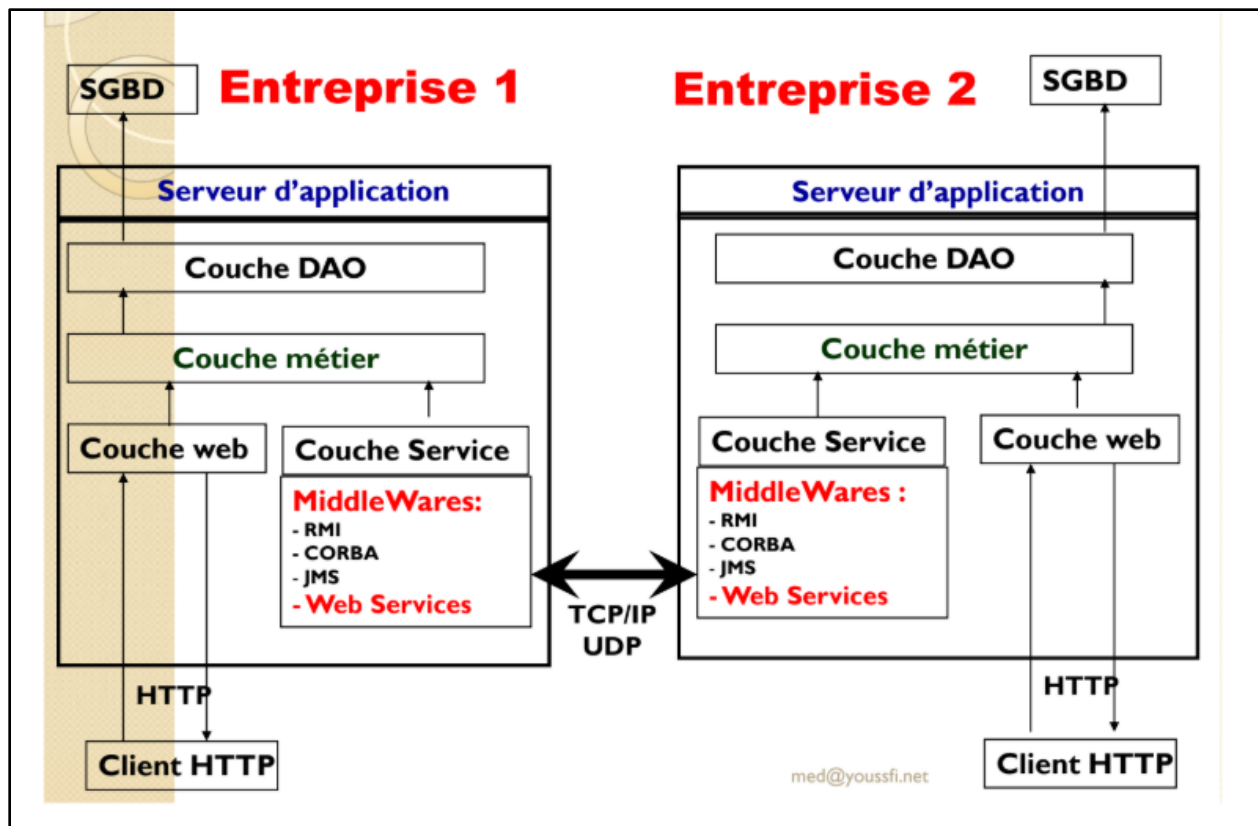
Step 2: Write the Logic

```
1  package services;
2
3  import java.util.ArrayList;
4  import java.util.Date;
5  import java.util.List;
6  import javax.xml.ws.Compte;
7  import javax.xml.ws.WebService;
8  import javax.xml.ws.WebMethod;
9  import javax.xml.ws.WebParam;
10
11 @WebService(name="BanqueWS")
12 public class BanqueServices {
13     @WebMethod
14     public double Convert(@WebParam(name="montant")double montant){
15         return montant*11.3;
16     }
17
18     @WebMethod
19     public Compte getCompte(@WebParam(name="code")Long code ){
20         return new Compte(code, Math.random() * 8000000, new Date());
21     }
22
23     @WebMethod
24     public List<Compte> getListCompte(){
25         List<Compte> cmpts = new ArrayList<>();
26         cmpts.add(new Compte(1L, Math.random() * 8000000, new Date()));
27         cmpts.add(new Compte(2L, Math.random() * 7500000, new Date()));
28         return cmpts;
29     }
30 }
```

Step 3 :



```
1  package server;
2
3  import javax.xml.ws.Endpoint;
4  import services.BanqueServices;
5
6  public class serverJws {
7
8      public static void main(String[] args) {
9
10         try {
11             String url = "http://localhost:8585/";
12
13             System.out.println("Starting Web Service...");
14
15             Endpoint.publish(url, new BanqueServices());
16
17             System.out.println("Web Service started successfully at: " + url);
18
19         } catch (Exception e) {
20             System.out.println("Error while starting the Web Service:"+e);
21             e.printStackTrace();
22         }
23     }
24 }
25 }
```



Internal Application Architecture

Each enterprise utilizes an **Application Server** that is organised into four primary layers:

- **Web Layer:** This layer acts as the entry point for external **HTTP Clients**, receiving requests via the **HTTP** protocol.
- **Service Layer:** This layer houses various **Middlewares** that facilitate distributed communication. The specific technologies listed include:
 - **RMI** (Remote Method Invocation)
 - **CORBA** (Common Object Request Broker Architecture)
 - **JMS** (Java Message Service)
 - **Web Services** (highlighted in red).
- **Business Layer:** Positioned in the middle of the stack, this layer processes core logic, receiving input from the Web and Service layers.
- **DAO (Data Access) Layer:** This top layer is responsible for managing data and communicating with an external **SGBD** (Database Management System).

Exam sol : Description of Points 1 to 6

1. **Point 1 (Client Call):** The **Client** application initiates a request by calling a method on the **STUB**. The **STUB** acts as a local proxy that represents the remote service on the client side.
2. **Point 2 (SOAP Request):** The **STUB** marshals (serializes) the method parameters into a **SOAP message** and sends it over the network to the server.
3. **Point 3 (Inferred - Server Processing):** Although not explicitly numbered in circles, the **SKELETON** receives the SOAP message, unmarshals it, and invokes the actual logic within the **BanqueService** on the server.
4. **Point 4 (Inferred - Service Return):** The **BanqueService** completes the operation and returns the result (e.g., a **Compte** object) to the **SKELETON**.
5. **Point 5 (SOAP Response):** The **SKELETON** packages the result into a **SOAP response** and transmits it back across the network to the client-side **STUB**.
6. **Point 6 (Return to Client):** The **STUB** unmarshals the SOAP response into a standard data type or object and returns it to the **Client** application, making the remote call appear like a local one.

Implementation of the Compte Class

Based on the class diagram provided in the source, which shows the attributes `-code : int` and `-solde : double`, here is a typical Java implementation suitable for a JAX-WS environment:

```
import jakarta.xml.bind.annotation.XmlRootElement;
import java.io.Serializable;

// The @XmlRootElement annotation allows the object to be
// converted to/from XML for the SOAP messages.
@XmlRootElement(name = "compte")
public class Compte implements Serializable {
    private int code;
    private double solde;

    // A default constructor is required for JAX-B serialization
    public Compte() {
    }

    public Compte(int code, double solde) {
        this.code = code;
        this.solde = solde;
    }

    // Getters and Setters
    public int getCode() {
        return code;
    }

    public void setCode(int code) {
        this.code = code;
    }

    public double getSolde() {
        return solde;
    }

    public void setSolde(double solde) {
        this.solde = solde;
    }
}
```

WSDL Structure (Web Services Description Language)

Element	Description
Types	Defines the data types used to describe exchanged messages
Messages	Abstract definition of data (names + types) transmitted between client and service
PortType	Defines a set of operations; each operation may have input, output, or fault messages
Binding	Links a portType to a concrete protocol (SOAP, HTTP, etc.)
Service	Specifies the service location (addresses of ports)
Port	Represents a service endpoint (network address + binding)
Operation	Defines an action provided by a service inside a port

Slide 18 explains the lifecycle of using a web service.

A client first searches for a web service in a UDDI directory. After finding the service, the client asks for its WSDL contract. The WSDL explains how the service must be called, including the operations and request format. Then the client sends an XML/SOAP request to the server, the server executes the request, and finally returns an XML response.